Universidad EIA · Fundamentos de Inteligencia Artificial · 2026-2



Predicción de abandono de clientes de telecomunicaciones

Clasificación de clientes con el conjunto Iranian Churn: el análisis original, que optimiza el F1, y su corrección con Recall y piso de precisión, con selección de variables, PCA y SHAP, más una aplicación web.

Juan Jose Jaramillo Mora
Sebastian Giraldo Franco
Martin Restrepo
Miguel Angel Zuleta
Julian Mora
Santiago Zuluaga

Informe final
16 de octubre de 2026

Resumen

Este trabajo aborda la predicción del abandono de clientes de una empresa de telecomunicaciones a partir de nueve meses de uso del servicio. Usamos el conjunto *Iranian Churn* de UCI, con 3.150 clientes, de los cuales el 15,71 % abandonó en el mes doce. Es una clasificación binaria con desbalance moderado: un modelo que siempre responda «permanece» alcanza una exactitud de 84,29 % sin detectar ni un solo abandono. Por eso la exactitud no decide nada y las métricas principales son las de la clase que abandona.

El trabajo reúne dos versiones del análisis, cada una en su propio cuaderno de Jupyter. El **cuaderno original** (`Prediccion_abandono_clientes_Original.ipynb`) sigue las doce etapas de un proyecto de analítica, compara 12 clasificadores con 4 estrategias de balanceo (41 combinaciones), ajusta los tres mejores y elige un Random Forest de 400 árboles con peso por clase y umbral de 0,50; optimiza el F1. El **cuaderno de corrección** (`Prediccion_abandono_clientes_Correccion.ipynb`) rehace el análisis con los cambios que pidió la profesora: ajusta primero los hiperparámetros de los doce modelos con los datos en crudo, prueba después sobre cada uno quince formas de usar las variables (doce métodos de selección y tres de PCA, con SHAP entre ellos) y elige por Recall con un piso de precisión de 0,60.

En el cuaderno original, el modelo elegido tiene un F1 de 0,8722 en validación cruzada y de 0,9038 en la muestra de prueba, con 94 de 99 abandonos detectados y 15 falsas alarmas entre 531 clientes que permanecen. En el cuaderno de corrección, la mejor de las 180 combinaciones es una red neuronal con las doce variables: Recall de 0,9546 y precisión de 0,6684 en validación anidada, y 95 de 99 abandonos detectados en la prueba con un umbral de 0,34, a costa de 44 falsas alarmas. En ninguno de los dos cuadernos quitar variables, seleccionarlas o resumirlas con PCA mejoró al modelo elegido.

Como la prueba tiene pocos abandonos, los resultados se reportan con su margen y no como cifras exactas. Concluimos que el modelo sirve para ordenar clientes por riesgo y priorizar contactos de retención, no como un sistema que dé certeza ni que prometa un ahorro. La elección entre las dos versiones depende de cuánto cueste una llamada de más frente a un cliente que se pierde, algo que este trabajo no estimó.

Palabras clave: abandono de clientes, telecomunicaciones, clasificación desbalanceada, Recall, F1, selección de variables, SHAP, scikit-learn.

Contenido

1 Introducción

2 El conjunto de datos

3 Partición de entrenamiento y prueba

4 Análisis exploratorio

5 Los dos cuadernos

6 Cuaderno original: preparación, escalado, ANOVA y PCA

7 Cuaderno original: modelado, 41 combinaciones

8 Cuaderno original: ajuste de hiperparámetros

9 Cuaderno original: selección del umbral

10 Cuaderno original: evaluación final

11 Cuaderno de corrección: lo que pidió la profesora

12 Cuaderno de corrección: ajuste de los doce modelos

13 Cuaderno de corrección: selección de variables, PCA y SHAP

14 Cuaderno de corrección: el mejor, el umbral y la evaluación final

15 Comparación de los dos cuadernos

16 Despliegue: variables, modelos guardados y aplicación web

17 Conclusiones, limitaciones y trabajo futuro

18 Referencias

A Anexo: reproducibilidad y estructura del repositorio

1. Introducción

Una empresa de telecomunicaciones en Irán pierde clientes frente a la competencia. Conseguir un cliente nuevo cuesta bastante más que retener a uno que ya se tiene, así que a la empresa le interesa saber quién está a punto de irse para contactarlo antes. El equipo de retención no puede llamar a todos los clientes: tiene capacidad para atender a unos pocos cada mes, así que lo que necesita es una lista corta de clientes en riesgo, ordenada por prioridad.

Los dos errores posibles no cuestan igual: dejar escapar a un cliente que sí se iba significa perderlo; llamar de más a uno que no pensaba irse solo gasta una llamada. Por eso el proyecto prioriza no dejar escapar clientes (recall) sin descuidar del todo acertar en las llamadas (precisión).

El problema se plantea como una clasificación binaria: para cada cliente se predice si abandona (1) o permanece (0). No se usa la exactitud para decidir nada, porque con 84,29 % de clientes que permanecen, un modelo que siempre responda «permanece» ya saca una exactitud muy alta sin detectar a nadie en riesgo. El cuaderno original mide el F1 de la clase abandono, que combina precisión y recall; el cuaderno de corrección optimiza el Recall con un piso de precisión (sección 11). El modelo además debe entregar una probabilidad por cliente, no solo una respuesta de sí o no, para poder ordenar la lista de contactos por prioridad.

Para anticipar el abandono se necesita información de cada cliente anterior al momento en que se va. Se usa el conjunto *Iranian Churn* de UCI, cuyas entradas corresponden a los primeros nueve meses de cada cliente, mientras que el abandono se registra al terminar el mes doce. El archivo no trae identificador de cliente ni fechas de cada medición: sin identificador no se puede saber si dos filas iguales son la misma persona o dos distintas, y sin fechas no se puede comprobar el orden exacto de los hechos. Estas dos limitaciones se tienen en cuenta en varias decisiones posteriores y se retoman en las limitaciones del trabajo (sección 17.2).

El análisis vive en dos cuadernos reproducibles, organizados como las etapas de un proyecto de analítica (entendimiento del negocio, enfoque analítico, requerimiento de datos, recolección, separación de entrenamiento y prueba, entendimiento de los datos, preparación, modelado, ajuste de hiperparámetros, umbral, evaluación final y despliegue). Uno es el análisis original y el otro la corrección de lo que la profesora nos señaló; la sección 5 explica en qué se diferencian. Las secciones 2 a 4 son comunes a los dos. Este informe describe además la aplicación web que publica el modelo.

2. El conjunto de datos

Se usa *Iranian Churn Dataset*, publicado en el repositorio de aprendizaje automático de UCI (identificador 563, licencia CC BY 4.0). Los datos provienen de una empresa de telecomunicaciones iraní y resumen, para cada cliente, el uso del servicio durante nueve meses junto con si abandonó o permaneció en el mes doce. El archivo original tiene 3.150 filas y 14 columnas (13 variables más la respuesta), sin valores faltantes. El archivo no se modifica en ningún momento; todo lo derivado se guarda con otro nombre.

Se encontraron 300 filas repetidas exactamente. Se conservan: sin identificador de cliente no se puede afirmar que sean errores de captura y no clientes distintos con el mismo comportamiento, así que cada fila se trata como un cliente distinto. La clase objetivo está desbalanceada: 2.655 clientes permanecieron y 495 abandonaron, es decir, 84,29 % frente a 15,71 %.

Tabla 1. Variables del conjunto de datos, en el orden en que se revisan.

Variable	Descripción	Tipo
`fallas_llamadas`	Llamadas fallidas	Numérica (conteo)
`quejas`	0 sin queja; 1 con queja (no cuenta cuántas)	Binaria
`antiguedad`	Meses de suscripción	Numérica
`nivel_cargo`	Nivel de cargo de 0 a 9 (hay valores de 10 en los datos)	Ordinal
`segundos_uso`	Segundos de llamadas	Numérica
`frecuencia_uso`	Número de llamadas	Numérica (conteo)
`frecuencia_sms`	Número de mensajes	Numérica (conteo)
`numeros_distintos`	Números distintos llamados	Numérica (conteo)
`grupo_edad`	Grupo de edad, de 1 a 5	Ordinal
`plan_tarifario`	1 pago por uso; 2 contractual	Nominal binaria
`estado`	1 línea activa; 2 no activa	Nominal binaria
`edad`	Edad; se elimina en la sección 4.5 por ser redundante con `grupo_edad`	Numérica
`valor_cliente`	Valor calculado por la empresa a partir de otras variables; UCI no publica su fórmula	Numérica derivada
`abandono`	1 si el cliente abandonó en el mes doce; 0 si permaneció	Objetivo

`estado` y `valor_cliente` merecen una revisión aparte, porque de la primera no se sabe con precisión qué cuenta como «activo» y de la segunda no se conoce la fórmula. Esa revisión, hecha ya con los datos de entrenamiento separados, se describe en la sección 4.6.

3. Partición de entrenamiento y prueba

Se separa un 20 % de prueba con `train_test_split`, estratificado por la variable objetivo y con semilla fija (`random_state=42`), **antes** de cualquier análisis exploratorio. La partición se hace sobre las trece variables de entrada originales (todavía con `edad`, que se elimina después en la sección 4.5, igual en los dos conjuntos) más la respuesta. Los dos cuadernos usan exactamente la misma partición.

Tabla 2. Tamaño y desbalance de cada conjunto.

Conjunto	Filas	Permanece (0)	Abandona (1)	% abandono
Completo	3.150	2.655	495	15,71 %
Entrenamiento	2.520	2.124	396	15,71 %
Prueba	630	531	99	15,71 %

Los tres coinciden en 15,71 % de abandono, gracias a la estratificación. A diferencia de un esquema con validación cruzada por grupos, aquí no se agrupan perfiles idénticos antes de partir: cada fila se trata como un cliente independiente (sección 2), así que puede haber el mismo perfil de entrada repartido entre entrenamiento y prueba. No se mide ni se reporta ese solapamiento; se deja como limitación explícita (sección 17.2).

> Regla que no se rompe en el resto del trabajo: desde aquí, toda decisión —exploración, preparación, modelado, ajuste, selección de variables y umbral— se toma únicamente con las 2.520 filas de entrenamiento. Los 630 clientes de prueba no se vuelven a tocar hasta la evaluación final de cada cuaderno (secciones 10 y 14), una sola vez cada una.

4. Análisis exploratorio

4.1 El desbalance

En entrenamiento, 2.124 clientes permanecieron y 396 abandonaron (15,71 %). Un modelo que siempre respondiera «permanece» acertaría la mayoría de las veces sin detectar a ninguno en riesgo: la razón de fondo para que la exactitud nunca decida en este proyecto.

4.2 Variables categóricas

Dos variables separan los grupos con claridad: abandona el 80,69 % de los 202 clientes con queja, frente al 10,05 % del resto; y el 47,44 % de los 624 clientes con línea no activa, frente al 5,27 % de los activos. El plan contractual abandona un 3,05 % frente al 16,79 % del plan de pago por uso. En `nivel_cargo` el abandono baja al subir el nivel y de 5 en adelante no hay ninguno, pero esos niveles tienen entre 6 y 24 clientes, así que no es una conclusión fuerte con tan poca muestra.

4.3 Variables numéricas

Las ocho variables numéricas tienen la media por encima de la mediana, señal de una cola larga a la derecha: las escalas también son muy distintas entre sí, desde `antiguedad` entre 3 y 47 hasta `segundos_uso`, que llega a 16.980; el escalado importará en la sección 6.

Quien abandona usa mucho menos el servicio: la mediana de `segundos_uso` es 1.188 frente a 3.594 entre quienes permanecen, la de `frecuencia_uso` es 25 frente a 64, la de `numeros_distintos` es 10 frente a 23 y la de `valor_cliente` es 97,82 frente a 267,94. `antiguedad` y `edad` dan casi igual en los dos grupos: la señal está en el uso, no en quién es el cliente.

4.4 Asociación de cada variable con el abandono (phik)

Se mide la relación entre variables con el coeficiente phik (φk), que funciona igual de bien para variables numéricas y categóricas, a diferencia de la correlación de Pearson, que solo sirve para números (Baak et al., 2020).

Tabla 3. Coeficiente phik de cada variable con el abandono, entrenamiento.

Variable	phik
`quejas`	**0,734**
`estado`	**0,705**
`segundos_uso`	**0,469**
`frecuencia_uso`	**0,445**
`valor_cliente`	**0,413**
`numeros_distintos`	**0,389**
`frecuencia_sms`	**0,325**
`antiguedad`	**0,267**
`nivel_cargo`	**0,262**
`plan_tarifario`	**0,152**
`grupo_edad` / `edad`	**0,105**
`fallas_llamadas`	**0,066**

4.5 Variables que repiten información

Hay 19 parejas de variables con phik de 0,60 o más. El caso extremo es `edad` y `grupo_edad`, con phik de 1,000: cada edad pertenece a un único grupo y cada grupo tiene una sola edad ($15 \leftrightarrow 1$, $25 \leftrightarrow 2$, $30 \leftrightarrow 3$, $45 \leftrightarrow 4$, $55 \leftrightarrow 5$), así que son la misma información escrita de dos maneras. Se elimina `edad` y se conserva `grupo_edad` por ser la forma documentada por UCI. Es la única variable que se elimina en todo el proyecto; quedan doce entradas más la

respuesta. Las variables de uso también se solapan bastante entre sí (`segundos_uso` con `frecuencia_uso`, 0,928; `frecuencia_sms` con `valor_cliente`, 0,918), pero cada una mide algo distinto y se conservan todas.

4.6 Estado y valor del cliente

Antes de confiar en estas dos variables dudosas se comprueba que de verdad aportan algo. De `estado` se revisa si distingue por su cuenta o solo repite lo que ya dicen las quejas: entre los 2.318 clientes que nunca presentaron ninguna queja, abandona el 37,04 % de los 494 no activos y el 2,74 % de los 1.824 activos, así que aporta información adicional. De `valor_cliente` se mide si es una combinación de las demás entradas con una regresión que intenta reconstruirlo: las otras once explican el 98 % de su variación, es decir, es casi una combinación matemática de las demás. Aun así se conserva: no introduce información de fuera del periodo medido.

> Queda un conjunto de doce entradas más `abandono`. Las señales más fuertes por phik son `quejas` y `estado`; el bloque de uso viene después; el perfil del cliente (edad, plan) casi no distingue nada por sí solo.

5. Los dos cuadernos

El proyecto tiene dos cuadernos porque se hicieron en dos momentos. El **cuaderno original** es el análisis completo que entregamos primero. La profesora lo revisó y pidió cambiar el orden y el criterio de decisión: no hacer la selección de variables ni el PCA primero, sino ajustar antes los hiperparámetros de todos los modelos con los datos en crudo y buscarlos con un método automático; probar después, sobre cada modelo ya ajustado, todos los métodos de selección de variables del cuaderno de clase, el PCA y SHAP; optimizar el Recall en lugar del F1; elegir el mejor de todas las combinaciones, probar con él todo lo anterior y validarlo; y mostrar la matriz de confusión de cada modelo. El **cuaderno de corrección** recoge esos cambios.

Los dos comparten los datos, la partición y las secciones de exploración (secciones 2 a 4 de este informe), y los dos se pueden ejecutar de forma independiente. La tabla resume en qué difieren; las secciones 6 a 10 describen el cuaderno original, las secciones 11 a 14 el de corrección y la sección 15 compara los resultados.

Tabla 4. Diferencias entre el cuaderno original y el cuaderno de corrección.

Aspecto	Cuaderno original	Cuaderno de corrección
Archivo	`Prediccion_abandono_clientes_Original.ipynb`	`Prediccion_abandono_clientes_Correccion.ipynb`
Métrica que se optimiza	F1 de la clase que abandona	Recall con piso de precisión de 0,60; el F1 desempata
Orden del trabajo	Compara 41 combinaciones con valores por defecto, elige los tres mejores y solo a ellos les ajusta los hiperparámetros	Ajusta primero los doce modelos con una búsqueda y después los cruza con los métodos de selección, PCA y SHAP
Hiperparámetros	`GridSearchCV` con rejillas pequeñas, solo para tres candidatos	`RandomizedSearchCV` en rangos amplios, 60 combinaciones por modelo (30 en Stacking), para los doce
Selección de variables	ANOVA con tres candidatos (12, 10 y 8 variables)	15 métodos por 12 modelos: 180 combinaciones, con PCA y SHAP
Matriz de confusión	Del modelo final y de otros dos candidatos	De los doce modelos ajustados
Elección del mejor	Regla del error estándar: el más rápido de los empatados	Mayor Recall con precisión sobre el piso; validación anidada
Umbral	0,50 (el de mayor F1)	0,34 (el de mayor Recall con el piso)
Modelo elegido	Random Forest con peso por clase	Red neuronal con las doce variables
Modelo guardado	`models/modelo_final.joblib`	`models/modelo_recall.joblib`
Relación con la aplicación	Es el modelo que carga `app.py` y el de la presentación	Incorpora las correcciones pedidas; documenta otra forma de elegir

6. Preparación: escalado, ANOVA y PCA

6.1 Escalado

Las variables están en escalas muy distintas y sus distribuciones están sesgadas (sección 4.3). Los modelos que miden distancias entre clientes (KNN, SVM, redes neuronales) quedarían dominados por las variables de números grandes si no se escala; a los modelos de árbol no les afecta, porque cortan cada variable por separado. El cuaderno original usa `MinMaxScaler`, que lleva cada variable al rango de 0 a 1: conserva la forma de la distribución y deja los ceros en cero, que aquí significan «cliente sin uso». El escalador se ajusta siempre con el entrenamiento y se aplica a los dos conjuntos, nunca al revés; en la comparación de modelos (sección 7) esto se repite dentro de cada una de las 10 particiones de validación cruzada. (El cuaderno de corrección no decide el escalador a mano: deja que la búsqueda pruebe `MinMaxScaler` y `StandardScaler` para cada modelo.)

6.2 Selección de variables con ANOVA

ANOVA compara, para cada variable, el promedio de quienes abandonan con el de quienes permanecen. Un valor de F alto indica que los dos grupos están lejos; un p pequeño, que esa distancia difícilmente se explica por azar.

Tabla 5. ANOVA (`f_classif`) de cada variable con el abandono, entrenamiento.

Variable	F	p
`quejas`	**968,57**	**0,0000**
`estado`	**839,51**	**0,0000**
`frecuencia_uso`	**259,48**	**0,0000**
`segundos_uso`	**252,24**	**0,0000**
`valor_cliente`	**229,31**	**0,0000**
`numeros_distintos`	**212,87**	**0,0000**
`frecuencia_sms`	**128,76**	**0,0000**
`nivel_cargo`	**108,20**	**0,0000**
`plan_tarifario`	**26,14**	**0,0000**
`antiguedad`	**4,49**	**0,0343**
`fallas_llamadas`	**0,68**	**0,4091**
`grupo_edad`	**0,11**	**0,7444**

El orden coincide con el de phik. Al final quedan dos variables sin relación individual aparente, `fallas_llamadas` y `grupo_edad` ($p > 0{,}05$). ANOVA mira cada variable sola: una F baja no demuestra que la variable sea inútil, puede aportar combinada con otras. Por eso la pregunta se comprueba con los modelos reales en la sección 7.4 y no se decide de antemano.

6.3 Análisis de componentes principales (PCA)

PCA se aplicó también, como alternativa a la selección de variables. A diferencia de ANOVA, que elige un subconjunto de las variables *originales* tal cual, PCA construye variables nuevas (componentes), cada una una combinación lineal de las doce originales, ordenadas de mayor a menor varianza explicada. Se escaló primero con `MinMaxScaler` (ajustado con entrenamiento), porque las componentes se construyen buscando la mayor variación y, sin escalar, las variables de números grandes dominarían el resultado solo por tener magnitudes mayores.

Tabla 6. Varianza acumulada por número de componentes de PCA.

Componentes	% varianza acumulada
1	38,11 %
2	52,74 %
4	74,43 %
6	89,37 %
7	93,68 % (pasa el 90 %)
8	96,70 % (pasa el 95 %)
12	100 %

La compresión es escasa: hacen falta 7 de las 12 variables originales para conservar el 90 % de la varianza y 8 para el 95 %. Se descartó PCA por dos razones. Primera, casi no comprime: pasar de doce variables a siete u ocho componentes no es una reducción que valga la pena, y el problema no tenía de entrada un problema de demasiadas variables. Segunda, y más importante: cada componente es una mezcla de las doce variables originales, así que un modelo entrenado sobre componentes no se podría explicar variable por variable. Si el equipo de retención pregunta por qué un cliente está en riesgo, hace falta poder responder «por sus pocas llamadas y su queja reciente», no «por la componente 3».

6.4 Decisión

La preparación queda en escalado con `MinMaxScaler`, sin PCA y sin descartar variables todavía por ANOVA: esa decisión se comprueba con los modelos reales en la sección 7.4, no se toma a priori.

7. Modelado: comparación de 41 combinaciones

7.1 Los doce modelos y las cuatro estrategias de balanceo

Se comparan los doce clasificadores vistos en el curso, con sus valores por defecto: ocho individuales (KNN, Naive Bayes, árbol de decisión, regresión logística, SVM, SVM lineal, red neuronal MLP y Random Forest) y cuatro ensambles (Bagging, AdaBoost, Gradient Boosting y Stacking, que combina Random Forest, SVM y KNN y decide entre ellos con una regresión logística). Se prueban cuatro formas de tratar el desbalance: sin balanceo (referencia), peso por clase (`class_weight="balanced"`, solo en cinco modelos que admiten ese parámetro), SMOTENC (crea clientes sintéticos de la clase minoritaria interpolando entre los reales, respetando que `quejas`, `plan_tarifario` y `estado` son categorías) y submuestreo aleatorio (descarta clientes de la mayoría hasta igualar los dos grupos). Con doce modelos por tres estrategias que todos admiten, más cinco de peso por clase, salen **41 combinaciones**.

7.2 Cómo se evita la fuga de información

Para cada una de las 10 particiones de validación cruzada estratificada: se separa la parte de entrenamiento de la que se evalúa; el escalador se ajusta solo con la parte de entrenamiento de esa vuelta y se aplica a las dos; y el balanceo (SMOTENC o submuestreo) se aplica únicamente a la parte de entrenamiento. Así el modelo nunca ve nada de la parte con la que se mide, ni siquiera a través del escalado o de clientes sintéticos construidos con información de la parte de evaluación.

7.3 Resultados de las 41 combinaciones

Tabla 7. Las ocho mejores de las 41 combinaciones, por F1 en validación cruzada.

Modelo	Estrategia	F1	Desv. F1	Precisión
Random Forest	**Peso por clase**	**0,8717**	**0,0399**	**0,8514**
Stacking	**SMOTENC**	**0,8702**	**0,0346**	**0,8578**
Random Forest	SMOTENC	0,8665	0,0368	0,8563
Stacking	Sin balanceo	0,8580	0,0265	0,8878
Red neuronal	**Sin balanceo**	**0,8498**	**0,0318**	**0,8756**
Red neuronal	SMOTENC	0,8432	0,0324	0,7658
Random Forest	Sin balanceo	0,8338	0,0341	0,8872
Bagging	Sin balanceo	0,8335	0,0265	0,8769

Los tres primeros están separados por 0,0052 de F1 (0,8717, 0,8702 y 0,8665), mucho menos que la desviación entre particiones (≈0,035): no hay un ganador claro todavía. La separación grande está entre familias de modelos: los árboles y sus conjuntos, junto con la red neuronal y KNN, pasan de 0,80; la regresión logística, los dos SVM, AdaBoost y Naive Bayes se quedan por debajo de 0,70. Con doce variables y relaciones que no son líneas rectas, los modelos que parten el espacio en regiones aprovechan mejor los datos: entre los 39 clientes que se quejaron y llaman poco (menos de 20 llamadas) abandonaron todos, y entre los 1.100 que no se quejaron y llaman mucho (61 o más) solo el 1,5 %. Un modelo lineal suma el efecto de cada variable por separado y no puede expresar directamente «se quejó y usó poco»; un árbol sí, porque puede partir primero por quejas y luego por uso.

Tabla 8. F1 promedio y mejor F1 por estrategia de balanceo, con el recall y la precisión promedio.

Estrategia	Combinaciones	F1 promedio	Mejor F1	Recall promedio	Precisión promedio
Sin balanceo	12	0,7217	0,8580	0,6947	0,8159
Peso por clase	5	0,7310	0,8717	0,8889	0,6406
SMOTENC	12	0,7463	0,8702	0,8906	0,6641
Submuestreo	12	0,7015	0,7772	0,9240	0,5727

SMOTENC tiene el mejor F1 promedio (0,7463). El submuestreo tiene el mejor recall promedio (0,9240) y el peor F1 y la peor precisión promedio (0,5727): detecta casi todos los abandonos a costa de muchas falsas alarmas, porque para igualar los dos grupos hay que descartar más del 80 % de los clientes que permanecen y el modelo se queda con pocos datos. El balanceo pesa distinto según el modelo: la regresión logística mejora bastante (de 0,5395 sin balanceo a 0,6582 con submuestreo), Random Forest mejora poco (de 0,8338 a 0,8717 con peso por clase), porque ya manejaba el desbalance razonablemente bien.

Sin balancear, la regresión logística, la SVM y la SVM lineal tienen precisión alta (entre 0,80 y 0,90) pero un recall de solo 0,41: avisan poco y casi siempre aciertan cuando avisan, que es lo contrario de lo que necesita el equipo de retención. Naive Bayes hace lo opuesto: recall de 0,9367 con precisión de 0,3723.

7.4 Los tres candidatos y si conviene quitar variables

Se toman las tres mejores combinaciones por F1, sin repetir modelo, para tener candidatos de familias distintas: Random Forest con peso por clase (0,8717), Stacking con SMOTENC (0,8702) y la red neuronal sin balancear (0,8498). Con estos tres candidatos se comprueba de verdad si conviene quitar variables con ANOVA (la pregunta que quedó abierta en la sección 6.2), entrenando cada uno con las doce variables completas, con las diez mejores y con las ocho mejores.

Tabla 9. F1 de los tres candidatos según el número de variables (selección ANOVA).

Modelo	12 variables	10 variables	8 variables
Stacking	**0,8702**	**0,8294**	**0,7932**
Random Forest	**0,8717**	**0,8172**	**0,7823**
Red neuronal	**0,8498**	**0,7734**	**0,6905**

Quitar variables empeora a los tres candidatos, y no por poco (la red neuronal cae de 0,85 a 0,69). Las dos variables que ANOVA dejaba últimas, `grupo_edad` y `fallas_llamadas`, son las primeras que se descartan al bajar a diez variables, y perderlas cuesta más de cinco centésimas de F1: por separado no distinguían a quienes abandonan, pero combinadas con las demás sí aportan. Se conservan las **doce variables completas**, sin selección.

8. Ajuste de hiperparámetros

8.1 Por qué el ajuste va con `Pipeline`

`GridSearchCV` entrena directamente el modelo que se le entrega, con los datos que uno le pasa: entre partir los datos y entrenar no hay dónde meter el escalado ni el balanceo. Si se aplicaran antes, sobre todo el entrenamiento, aprenderían también de las filas que después se usan para evaluar. Con SMOTENC el problema es mayor, porque crea clientes nuevos interpolando entre los que abandonaron: si se aplicara antes de partir en pliegues, algunos sintéticos saldrían de clientes que después se evalúan, y además las particiones de evaluación quedarían con una proporción de abandono irreal. La solución es entregarle a la búsqueda un `Pipeline` de imbalanced-learn (Lemaître et al., 2017) que lleve los pasos adentro: al entrenar, el escalado y el balanceo trabajan solo con la parte de entrenamiento de esa vuelta; al evaluar, el escalador aplica lo que ya aprendió y el balanceo no hace nada.

8.2 Las tres búsquedas

Tabla 10. Ajuste de hiperparámetros de los tres candidatos.

Modelo	Qué se varió	Mejor configuración	F1 por defecto	F1 ajustado
Random Forest	Nº de árboles (100, 200, 400), profundidad (sin límite o 10), mínimo por hoja (1 o 2)	400 árboles, sin límite de profundidad, hoja mínima 1	0,8717	0,8722
Stacking	Árboles del RF interno (100 o 200), vecinos del KNN interno (5 o 9)	100 árboles, 9 vecinos	0,8702	0,8734
Red neuronal	Forma de las capas ocultas ((100,), (50,), (100, 50)), alpha (0,0001 o 0,001)	capas (100, 50), alpha 0,0001	0,8498	0,8721

Todas se buscan con `GridSearchCV(scoring="f1")` y las mismas 10 particiones de la sección 7. Las rejillas son pequeñas a propósito: cada configuración se evalúa diez veces y Stacking tarda unos segundos en cada una. A la red neuronal se le subió el máximo de iteraciones a 2.000, porque en la sección 7 varias particiones no habían terminado de converger con 1.000.

8.3 El modelo elegido

Los tres candidatos ajustados quedaron a milésimas de F1, muy por debajo de la desviación entre particiones (≈0,03-0,04). Elegir el número más alto sería casi azar: con otra semilla de particiones el orden cambiaría. Se aplica en su lugar una regla conocida:

entre las configuraciones que caen dentro de un error estándar de la mejor (la desviación entre particiones dividida por la raíz de 10), se elige la más rápida de entrenar.

Tabla 11. F1, desviación, error estándar y costo de ajuste de los tres candidatos.

Modelo	F1	Desviación	Error estándar	Segundos por ajuste
Random Forest	**0,8722**	**0,0326**	**0,0103**	**0,51**
Stacking	0,8734	0,0418	0,0132	1,40
Red neuronal	0,8721	0,0348	0,0110	1,04

El mejor F1 es 0,8734 y su margen de un error estándar llega hasta 0,8602: los tres caen dentro, así que se elige el más rápido de entrenar, **Random Forest**. Además de la velocidad, pesan otras tres razones: es un solo modelo, mientras Stacking entrena tres clasificadores más una regresión logística que decide entre ellos; permite saber qué variables pesaron en cada predicción (sección 16.1), lo que el equipo de retención necesita para preparar la llamada, el mismo argumento con el que se descartó PCA en la sección 6.3; y con peso por clase no hay remuestreo, así que no hay clientes sintéticos que explicar ni decisiones sobre qué variables son categóricas para SMOTENC.

> Modelo elegido: Random Forest, 400 árboles, sin límite de profundidad, hoja mínima 1, peso por clase (`class_weight="balanced"`), dentro de un `Pipeline` con `MinMaxScaler`.

9. Selección del umbral

El modelo no solo dice sí o no: entrega una probabilidad de abandono. De esa probabilidad depende a partir de qué valor se da la alarma. El 0,5 es el valor por defecto, pero no tiene nada de especial, y con clases desbalanceadas no siempre es el que da el mejor F1; por eso se recorre una grilla de umbrales y se elige con entrenamiento. Las salidas del modelo sirven como puntuaciones para clasificar y priorizar clientes; no se interpretan como riesgos exactos. La prueba sigue sin abrirse.

9.1 Probabilidades fuera de pliegue

Se necesitan probabilidades que el modelo haya dado sin haber visto a ese cliente al entrenar. Se usa `cross_val_predict` con las mismas diez particiones: cada uno de los 2.520 clientes recibe la probabilidad que le asignó un modelo entrenado sin él. El promedio de esas probabilidades es 0,1865, por encima del 0,1571 de abandono real: el modelo exagera algo el riesgo, esperable porque el peso por clase lo empuja a favor de la minoría. El cuaderno original no aplica ninguna calibración de probabilidades; las usa tal como salen para ordenar a los clientes.

9.2 El umbral

Se recorren los umbrales de 0,05 a 0,95, de cinco en cinco centésimas, y se mide el F1, la precisión y el recall de cada uno sobre las probabilidades fuera de pliegue.

Tabla 12. F1, precisión y recall según el umbral, sobre entrenamiento (2.520 clientes).

Umbral	F1	Precisión	Recall	Alertas
0,05	0,6003	0,4308	0,9899	910
0,20	0,7505	0,6082	0,9798	638
0,30	0,8219	0,7146	0,9672	536
0,40	0,8529	0,7827	0,9369	474
0,45	0,8683	0,8188	0,9242	447
0,50	0,8742	0,8463	0,9040	423
0,55	0,8697	0,8632	0,8763	402
0,60	0,8626	0,8773	0,8485	383
0,70	0,8200	0,8834	0,7652	343
0,80	0,7645	0,9007	0,6641	292

El F1 más alto está en el umbral 0,50, con 0,8742, y coincide con el corte predeterminado. La curva es bastante plana entre 0,45 y 0,60, todos por encima de 0,86, así que moverse dentro de ese rango cambia poco. El intercambio se ve con claridad en los extremos: con

umbral 0,05 el recall es 0,9899 pero la precisión 0,4308, y de 910 alertas más de la mitad serían clientes que no pensaban irse; con 0,95 la precisión sube a 0,9592 y el recall cae a 0,3561.

> Umbral elegido: 0,50. Con él, la precisión es 0,8463, el recall 0,9040 y alertaría sobre 423 de los 2.520 clientes de entrenamiento (16,8 %). Si el equipo de retención tuviera menos capacidad, el umbral es la perilla para ajustar cuántas alertas salen, a cambio de dejar escapar más clientes. Este resultado se usa para seleccionar el corte, no como evaluación independiente.

10. Evaluación final

Se abren los 630 clientes de prueba reservados en la sección 3, que no han participado en ninguna decisión: ni en la exploración, ni en la comparación de modelos, ni en el ajuste, ni en la elección del umbral. Modelo y umbral ya están congelados. Este conjunto ya se había consultado en una versión anterior del cuaderno, así que la medición documenta el cambio de criterio y no constituye una prueba nueva intacta; no se usa para modificar nada.

Se entrena con las 2.520 filas de entrenamiento completas el modelo elegido en la sección 8 y se predice sobre los 630 de prueba con el umbral 0,50.

Tabla 13. Métricas del modelo final del cuaderno original en la muestra de prueba (630 clientes, 99 abandonos).

Métrica	Valor
F1 del abandono	**0,9038**
Recall	**0,9495**
Precisión	**0,8624**
Exactitud	**0,9683**

El F1 de prueba (0,9038) queda por encima del 0,8722 de validación cruzada y del 0,8742 de la selección del umbral. No debe leerse como que el modelo resultó mejor de lo esperado: se entrena con 2.520 filas en vez de las 2.268 de cada vuelta de validación, y sobre todo 630 clientes con 99 abandonos son pocos, así que esta cifra tiene su propio margen de varias centésimas. Lo razonable es decir que las dos mediciones son consistentes y que el desempeño está entre 0,87 y 0,90. La exactitud (0,9683) es la métrica que se decidió no usar para decidir, y aquí se ve por qué: un modelo que respondiera siempre «permanece» sacaría 0,8429 sin detectar a nadie.

Tabla 14. Matriz de confusión del modelo final del cuaderno original en la muestra de prueba.

	El modelo alerta	El modelo no alerta
Abandonó (real)	**94 detectados**	**5 no detectados**
Permaneció (real)	**15 falsas alarmas**	**516 bien descartados**

De los 99 clientes que abandonaron, el modelo detectó 94 y dejó escapar 5. De los 531 que permanecieron, alertó de más sobre 15. En términos de la operación, la lista tendría 109 nombres, 94 de ellos clientes que de verdad se iban; el costo son 15 llamadas gastadas en gente que no pensaba irse, y el riesgo que queda son 5 clientes que se pierden sin aviso.

Para comparar, el cuaderno original muestra también las matrices de los otros dos candidatos, sin ajustar sus umbrales con la prueba (se usa su decisión predeterminada de 0,50): Stacking con SMOTENC detecta 92 de los 99 abandonos con 12 falsas alarmas, y la red

neuronal sin balanceo detecta 82 con 10 falsas alarmas. Se mantiene la elección de Random Forest, que detecta más abandonos con una diferencia pequeña en falsas alarmas.

10.1 Priorizar contactos

Para el uso previsto, decidir a quién llamar primero, importa más el orden por probabilidad que el umbral exacto. Se ordenan los 630 clientes de prueba por probabilidad y se mide cuántos abandonos reales caen en los primeros puestos.

Tabla 15. Qué pasa si solo se contacta a una fracción de los clientes de mayor riesgo (cuaderno original).

% contactado	Clientes	Precisión (aciertos)	Recall (abandonos cubiertos)	Veces mejor que al azar
5 %	32	90,62 %	29,29 %	5,77×
10 %	63	92,06 %	58,59 %	5,86×
15 %	94	90,43 %	85,86 %	5,75×
20 %	126	76,19 %	96,97 %	4,85×
25 %	158	61,39 %	97,98 %	3,91×
30 %	189	51,32 %	97,98 %	3,27×

Contactando al 15 % de los clientes (94 personas) se alcanza al 85,86 % de los que abandonan y acierta el 90,43 % de las llamadas: 5,75 veces lo que daría llamar al azar. A partir del 20 % el rendimiento cae: se capturan casi todos los abandonos (96,97 %), pero ya solo tres de cada cuatro llamadas aciertan, y las últimas llamadas rinden cada vez menos.

11. Lo que pidió la profesora y cómo cambia el método

11.1 Los cambios pedidos

- No hacer primero la selección de variables ni el PCA: primero se ajustan los hiperparámetros de todos los modelos con los datos en crudo y se obtiene el mejor de cada uno.
- Los hiperparámetros los encuentra una búsqueda, no se fijan a mano, y con rangos amplios.
- Después, con cada modelo ya ajustado, se prueban todos los métodos de selección de variables del cuaderno de clase, el PCA y SHAP, en todas las combinaciones.
- La métrica que se optimiza es el Recall.
- De la matriz de combinaciones se elige el mejor de todos, se prueba todo con ese y luego se valida.
- Se muestra la matriz de confusión de cada modelo.

11.2 El Recall y el piso de precisión

El Recall responde cuántos de los clientes que se van alcanzamos a detectar. Usado solo tiene un problema: un modelo que respondiera que todos los clientes abandonan tendría un Recall de 1,00. En entrenamiento ese modelo tiene una precisión de 0,1571, es decir, habría que llamar a los 2.520 clientes y solo el 15,7 % se iba. Por eso la búsqueda maximiza el Recall entre las configuraciones que mantienen una precisión mínima, el **piso de precisión**, fijado en 0,60: seis de cada diez alertas deben ser clientes que de verdad se iban. Entre las configuraciones que cumplen el piso se elige la de mayor Recall, con el F1 como desempate; si ninguna lo cumple, se toma la de mayor precisión.

11.3 Cómo se arma cada modelo

Cada modelo va en un `Pipeline` de imbalanced-learn con cuatro pasos: escalador, balanceo, selección de variables y modelo. El balanceo va antes que la selección porque SMOTENC necesita conocer la posición de las tres variables categóricas entre las doce columnas. Todos los pasos se ajustan solo con la parte de entrenamiento de cada partición de validación cruzada. En la búsqueda, el escalador (`MinMaxScaler` o `StandardScaler`), el balanceo (ninguno, SMOTENC o submuestreo) y el peso por clase son parámetros que se prueban junto con los hiperparámetros del modelo.

12. Ajuste de los doce modelos en crudo

12.1 La línea base y la búsqueda

Primero se miden los doce modelos con sus valores por defecto, con las doce variables y sin balancear: es la línea base. Con ella, Naive Bayes encabeza con un Recall de 0,9367, pero su precisión es de solo 0,3723 y no cumple el piso. Entre los que sí lo cumplen lidera Stacking con 0,8308, seguido de KNN (0,8083), Bagging (0,7956), Random Forest (0,7906) y el árbol (0,7803); los de abajo casi no detectan abandonos: la SVM lineal (0,4094), la regresión logística (0,4095) y la SVM (0,4147).

Después, una búsqueda aleatoria (`RandomizedSearchCV`, Bergstra y Bengio, 2012) sortea 60 combinaciones por modelo dentro de rangos amplios (30 en el Stacking, que solo tiene tres parámetros que buscar), con las mismas diez particiones y el criterio de la sección 11.2 para escoger. Los parámetros que solo acotan el tiempo de cómputo (iteraciones máximas de la SVM, de la SVM lineal y de la red neuronal, y el solver `liblinear` de la regresión logística) van como opción única. El Stacking se arma con el Random Forest, la SVM y el KNN ya ajustados como modelos base.

Tabla 16. Los doce modelos ajustados, con su Recall por defecto y ajustado (validación cruzada de 10 particiones).

Modelo	Escalador	Balanceo	Recall por defecto	Recall ajustado	Desv. Recall	Precisión	Cumple el piso
Red neuronal	MinMax	submuestreo	0,6314	0,9597	0,0297	0,6762	Sí
SVM	Standard	ninguno	0,4147	0,9546	0,0387	0,6707	Sí
Stacking	Standard	submuestreo	0,8308	0,9546	0,0309	0,6759	Sí
Gradient Boosting	Standard	submuestreo	0,7578	0,9545	0,0427	0,6530	Sí
Bagging	Standard	submuestreo	0,7956	0,9520	0,0367	0,6173	Sí
Random Forest	MinMax	submuestreo	0,7906	0,9520	0,0421	0,6078	Sí
Naive Bayes	MinMax	ninguno	0,9367	0,9367	0,0322	0,3728	No
KNN	Standard	submuestreo	0,8083	0,9342	0,0448	0,6224	Sí
Árbol de decisión	Standard	submuestreo	0,7803	0,9015	0,0385	0,6377	Sí
AdaBoost	MinMax	ninguno	0,5738	0,7506	0,0816	0,7832	Sí
Regresión logística	Standard	ninguno	0,4095	0,6519	0,0806	0,6586	Sí
SVM lineal	Standard	ninguno	0,4094	0,4121	0,0836	0,8791	Sí

La red neuronal queda primera con un Recall de 0,9597 y una precisión de 0,6762. Detrás vienen la SVM y el Stacking con 0,9546, Gradient Boosting con 0,9545 y Bagging y Random Forest con 0,9520: seis modelos pasan de 0,95 con la precisión sobre el piso. El ajuste

cambia mucho el orden de la línea base: la SVM sube de 0,4147 a 0,9546 y la red neuronal de 0,6314 a 0,9597. Siete modelos eligieron submuestreo aleatorio y cinco ninguno; ninguno eligió SMOTENC. Ocho usan el escalador estándar y cuatro el de rango 0 a 1. Naive Bayes sigue sin cumplir el piso con ninguna configuración (la tabla muestra la de mayor precisión que encontró la búsqueda). La regresión logística (0,6519), la SVM lineal (0,4121) y AdaBoost (0,7506) cumplen el piso, pero se quedan muy por debajo de los demás. Estas cifras se midieron con las mismas particiones con las que se eligió cada configuración, así que son algo optimistas.

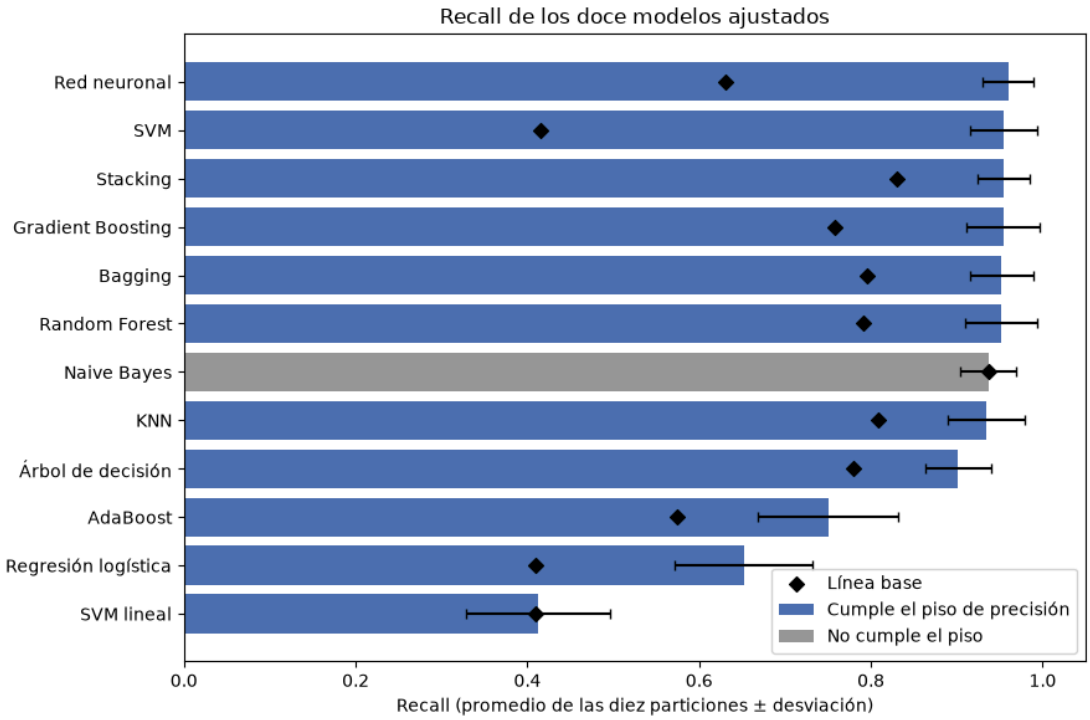


Figura 1. Recall de los doce modelos ajustados con su desviación entre particiones. El rombo negro marca la línea base y en gris están los modelos que no alcanzan el piso de precisión.

12.2 Sobreajuste en KNN y en el árbol

Recorriendo el número de vecinos del KNN y la profundidad del árbol, con el resto de la configuración ajustada fija, se ve que el KNN pierde Recall de forma gradual al subir los vecinos y que su precisión cae por debajo del piso desde cerca de 10 vecinos; la búsqueda eligió 8, justo antes de cruzarlo (Recall de 0,9342 y precisión de 0,6224). El árbol se separa de su validación desde profundidad 5 y se estabiliza pasando de 11: con la profundidad elegida el Recall es de alrededor de 0,96 en entrenamiento y 0,90 en validación, un sobreajuste moderado.

12.3 Matriz de confusión de cada modelo

Con predicciones fuera de pliegue de las diez particiones (cada cliente predicho por un modelo que no lo vio) se arma la matriz de confusión de los doce modelos ajustados. Con el abandono como clase positiva, VP son los abandonos detectados, FN los que se escaparon,

FP las alertas falsas y VN las permanencias bien clasificadas.

Tabla 17. Matriz de confusión de los doce modelos ajustados, sobre el entrenamiento (396 abandonos y 2.124 permanencias).

Modelo	VP	FN	FP	VN	Recall	Precisión
Red neuronal	**380**	**16**	**183**	**1.941**	**0,9596**	**0,6750**
Stacking	378	18	184	1.940	0,9545	0,6726
SVM	378	18	187	1.937	0,9545	0,6690
Gradient Boosting	378	18	206	1.918	0,9545	0,6473
Bagging	377	19	239	1.885	0,9520	0,6120
Random Forest	377	19	248	1.876	0,9520	0,6032
Naive Bayes	371	25	629	1.495	0,9369	0,3710
KNN	370	26	229	1.895	0,9343	0,6177
Árbol de decisión	357	39	208	1.916	0,9015	0,6319
AdaBoost	297	99	84	2.040	0,7500	0,7795
Regresión logística	258	138	134	1.990	0,6515	0,6582
SVM lineal	163	233	23	2.101	0,4116	0,8763

La red neuronal detecta 380 de los 396 abandonos con 183 falsas alertas. El Stacking y la SVM detectan 378 con 184 y 187 falsas alertas. Naive Bayes detecta 371 pero genera 629 falsas alertas, más de seis de cada diez alertas equivocadas. La SVM lineal solo detecta 163 de los 396 abandonos y la regresión logística 258.

13. Selección de variables, PCA y SHAP: la matriz de 180 combinaciones

Con los doce modelos ya ajustados se prueba cada uno con quince formas de usar las variables: todas, doce métodos de selección y tres variantes de PCA. Salen 180 combinaciones, cada una evaluada con las mismas diez particiones. Dentro del entrenamiento se separa una parte interna de validación (75 % y 25 %) que sirve únicamente para armar los subconjuntos de variables y para elegir cuántas conservar con cada modelo; la prueba no se toca.

13.1 Los quince métodos

Los métodos son los del cuaderno de selección de rasgos de clase, más SHAP y PCA. Los que son transformadores de scikit-learn se reajustan *dentro* del `Pipeline` en cada partición, y sus cifras no tienen sesgo de selección. Siete de las quince variantes (varianza baja, correlación, selección hacia adelante, RFECV, L1 y las dos de SHAP) solo entregan una lista de variables armada con la parte interna del entrenamiento; esa lista se aplica igual en todas las particiones, y como la parte interna se usa luego en la validación cruzada, sus cifras pueden salir algo optimistas. En los métodos con k variable, cada modelo se queda con el k de mayor Recall que cumple el piso en la validación interna, y si dos k empatan gana el más pequeño. La selección hacia adelante y RFECV se puntúan con F1 y no con Recall: sin un piso, el Recall solo premiaría al subconjunto que alerta a todos.

Tabla 18. Los quince métodos de la matriz.

Método	Dónde se ajusta	Cuántas variables
Todos los rasgos	—	Las doce variables
ANOVA (`SelectKBest`)	Dentro del Pipeline	k de 1 a 11, elegido por modelo
Chi cuadrado	Dentro del Pipeline	k de 1 a 11; puntaje sobre variables en rango 0 a 1
Información mutua	Dentro del Pipeline	k de 1 a 11
Varianza baja	Lista fija	Umbral 0,01; conserva las doce
Selección hacia adelante	Lista fija	k de 1 a 11 (regresión logística, F1)
RFECV	Lista fija	Automático; elige 10 variables
Regularización L1	Lista fija	C de 0,1; deja fuera `segundos_uso` y `valor_cliente`
Importancia de Gradient Boosting	Dentro del Pipeline	k de 1 a 11
SHAP, k mejores	Lista fija	k de 1 a 11 sobre el ranking SHAP de Gradient Boosting
SHAP, 95 % acumulado	Lista fija	Ocho variables
PCA, 95 % de la varianza	Dentro del Pipeline	Componentes que llegan al 95 %
PCA con k componentes	Dentro del Pipeline	k de 1 a 12, elegido por modelo
Correlación entre variables	Lista fija	Quita `segundos_uso` y `frecuencia_sms`
PCA de dos componentes	Dentro del Pipeline	Dos componentes

El ranking de SHAP sale de un solo modelo, el Gradient Boosting ajustado en la sección 12, y lo encabezan `estado` (2,0492) y `quejas` (1,1689); junto con `frecuencia_uso` y `segundos_uso` reúnen el 62,18 % de la importancia, y `plan_tarifario` cierra con 0,0141. El filtro de correlación encuentra dos parejas, `segundos_uso` con `frecuencia_uso` (0,946) y `frecuencia_sms` con `valor_cliente` (0,926).

13.2 Resultados

De las 180 combinaciones, 145 cumplen el piso de precisión. El mapa de calor muestra el Recall de cada una; las celdas grises no alcanzan el piso y su Recall no cuenta.

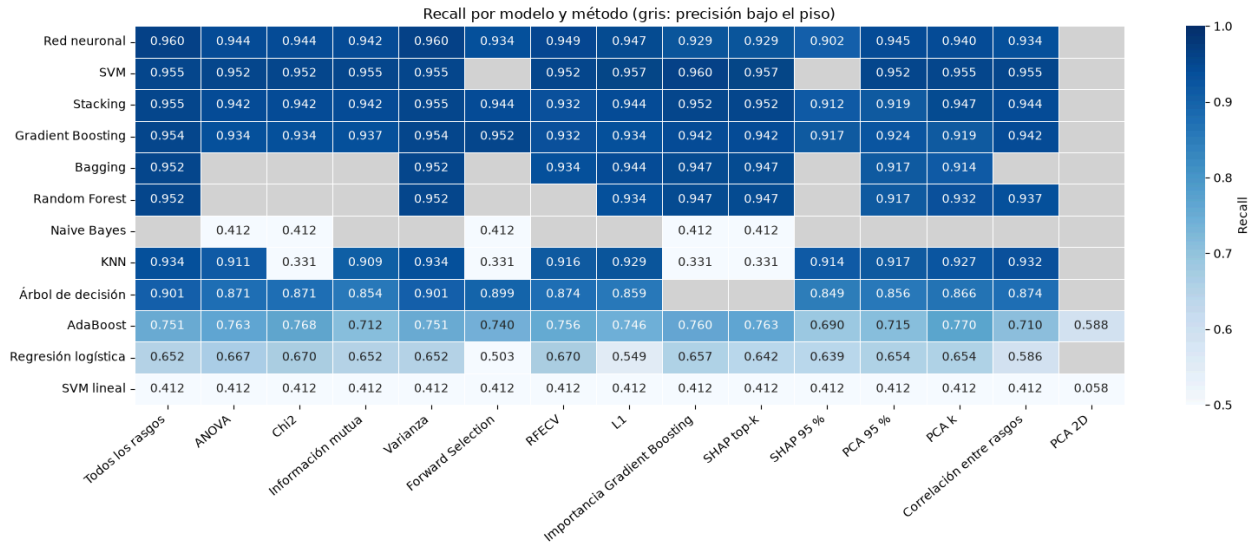


Figura 2. Recall por modelo y método (gris: precisión por debajo del piso).

Tabla 19. La mejor combinación de cada modelo (mayor Recall con precisión sobre el piso) frente a usar todas las variables. En los métodos de PCA, «variables» cuenta componentes.

Modelo	Mejor método	Variables	Recall	Precisión	Recall con todas
Red neuronal	Todos los rasgos	12	0,9597	0,6762	0,9597
SVM	Importancia Gradient Boosting	11	0,9597	0,6731	0,9546
Stacking	Todos los rasgos	12	0,9546	0,6759	0,9546
Gradient Boosting	Todos los rasgos	12	0,9545	0,6530	0,9545
Bagging	Todos los rasgos	12	0,9520	0,6173	0,9520
Random Forest	Todos los rasgos	12	0,9520	0,6078	0,9520
Naive Bayes	ANOVA	1	0,4121	0,8078	0,9367
KNN	Todos los rasgos	12	0,9342	0,6224	0,9342
Árbol de decisión	Todos los rasgos	12	0,9015	0,6377	0,9015
AdaBoost	PCA k	12	0,7704	0,8425	0,7506
Regresión logística	Chi2	11	0,6697	0,6632	0,6519
SVM lineal	PCA 95 %	9	0,4121	0,8821	0,4121

El mejor Recall es 0,9597 y lo alcanzan tres combinaciones: la red neuronal con todas las variables (precisión 0,6762), la misma red con el filtro de varianza baja, que no quita ninguna de las doce y por eso es idéntica, y la SVM con la importancia de Gradient Boosting (11 variables, precisión 0,6731). Quitar variables casi nunca ayuda: en siete de los doce modelos (red neuronal, Stacking, Gradient Boosting, Bagging, Random Forest, KNN y árbol) la mejor combinación es la de todas las variables. La selección sí mejora algo en la SVM, en AdaBoost (de 0,7506 a 0,7704 con PCA de 12 componentes) y en la regresión logística (de 0,6519 a 0,6697 con chi cuadrado), pero son diferencias menores que la

desviación entre particiones. Naive Bayes solo cumple el piso con combinaciones de una variable, con Recall de 0,4121. PCA con dos componentes casi nunca cumple el piso: solo AdaBoost y la SVM lineal quedan sobre él, con Recall de 0,588 y 0,058.

Las listas fijas son algo optimistas y aun así ninguna supera a la columna de todas las variables; la conclusión de que seleccionar no sube el Recall se sostiene. En las curvas por número de variables, desde unas cinco o seis la mayoría de los modelos se aplana cerca de su nivel con todas.

Tabla 20. Cuántas de las 132 selecciones posibles (doce modelos por once métodos con lista de variables) conservan cada variable.

Variable	Selecciones	Porcentaje
`quejas`	**132**	**100,0 %**
`estado`	**129**	**97,7 %**
`frecuencia_uso`	**122**	**92,4 %**
`antiguedad`	**118**	**89,4 %**
`numeros_distintos`	**110**	**83,3 %**
`valor_cliente`	**108**	**81,8 %**
`frecuencia_sms`	**106**	**80,3 %**
`segundos_uso`	**99**	**75,0 %**
`nivel_cargo`	**94**	**71,2 %**
`grupo_edad`	**84**	**63,6 %**
`fallas_llamadas`	**81**	**61,4 %**
`plan_tarifario`	**73**	**55,3 %**

`quejas` queda en todas las selecciones y `estado` en 129; las que menos aparecen son `plan_tarifario` (73), `fallas_llamadas` (81) y `grupo_edad` (84). El ranking de SHAP coincide: estado y quejas encabezan.

14. El mejor de todos, el umbral y la evaluación final

14.1 La elección

Se toma la combinación de mayor Recall con precisión sobre el piso. Las de PCA no compiten: sus componentes no son variables que se puedan explicar al equipo de retención. Si hay un empate exacto de Recall, gana la de mayor precisión. La elegida es la **red neuronal de dos capas (32 y 16 neuronas) con las doce variables**, escalador de rango 0 a 1 y submuestreo aleatorio: Recall de 0,9597 y precisión de 0,6762 en la matriz. Hay 23 combinaciones sin PCA a menos de un error estándar (0,0094) del mejor, y la SVM con la importancia de Gradient Boosting empata en Recall con 11 variables (precisión de 0,6731). La diferencia entre ellas es menor que la desviación entre particiones, así que la elección entre las empatadas no es firme; se escoge la red neuronal porque alcanza el mismo Recall con la precisión más alta y devuelve probabilidades.

14.2 Reajuste y validación anidada

Con la selección ya fija se vuelve a buscar el modelo en el mismo espacio, con otra semilla, para que el sorteo de combinaciones no repita el anterior. El reajuste no mejoró: Recall de 0,9572 con precisión de 0,6085, apenas sobre el piso, así que se conserva la configuración de la matriz.

La cifra que sí se puede creer es la de la validación anidada (Varma y Simon, 2006): en cada una de las diez particiones externas se busca el mejor modelo con cinco particiones internas (escalador, balanceo e hiperparámetros incluidos) y se mide en la parte externa, que la búsqueda nunca vio. La selección de variables queda fija, así que lo único que esta validación no corrige es esa decisión.

Tabla 21. Recall del modelo elegido según cómo se mida.

Medición	Recall	Desv. Recall	Precisión	F1
Matriz de la sección 13	0,9597	0,0297	0,6762	0,7931
Reajuste de la elección	0,9572	0,0267	0,6085	0,7431
Validación anidada	0,9546	0,0260	0,6684	0,7843

La validación anidada da un Recall de 0,9546 con precisión de 0,6684, frente a 0,9597 y 0,6762 de la matriz: elegir con las mismas particiones infló el Recall en unos 0,005, poco.

14.3 SHAP del ganador

SHAP (Lundberg y Lee, 2017) mide cuánto aporta cada variable a la predicción de un modelo. Como el modelo elegido no es de árboles, se usa una explicación sobre su probabilidad con una muestra de fondo de 50 clientes, calculada sobre 200 clientes de

entrenamiento.

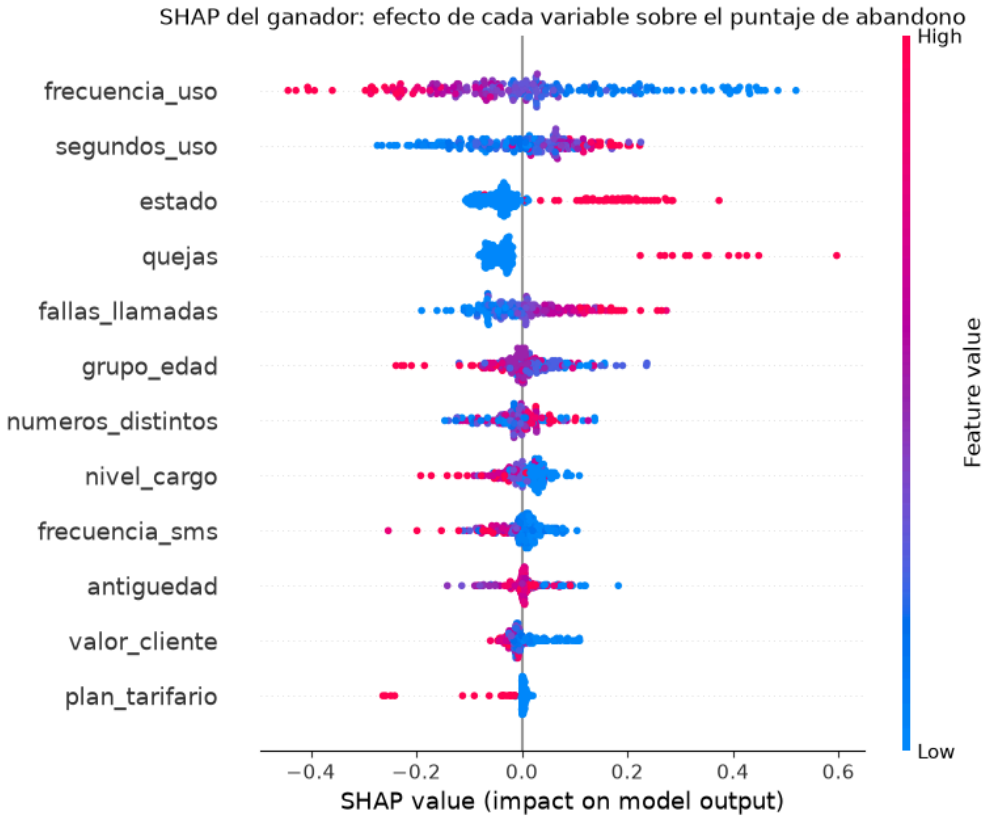


Figura 3. Efecto de cada variable sobre el puntaje de abandono del modelo elegido. Cada punto es un cliente; el color indica el valor de la variable.

Tabla 22. Importancia media (valor absoluto de SHAP) de cada variable en el modelo elegido.

Variable	Importancia media	Acumulada
`frecuencia_uso`	0,1521	23,7 %
`segundos_uso`	0,0864	37,2 %
`estado`	0,0777	49,4 %
`quejas`	0,0647	59,5 %
`fallas_llamadas`	0,0633	69,3 %
`grupo_edad`	0,0448	76,3 %
`numeros_distintos`	0,0343	81,7 %
`nivel_cargo`	0,0318	86,7 %
`frecuencia_sms`	0,0306	91,4 %
`antiguedad`	0,0223	94,9 %
`valor_cliente`	0,0215	98,3 %
`plan_tarifario`	0,0111	100,0 %

`frecuencia_uso` lidera con 0,1521, que es el 23,7 % de la importancia, seguida de `segundos_uso` (0,0864), `estado` (0,0777) y `quejas` (0,0647); las cuatro reúnen el 59,5 % del total. En el gráfico empujan hacia el abandono una frecuencia de uso baja, un estado alto (cliente no activo), tener quejas y más fallas de llamadas. El orden es distinto del que dio Gradient Boosting en la sección 13, con estado y quejas primero, porque la importancia depende del modelo. Además `frecuencia_uso` y `segundos_uso` se correlacionan en 0,946 y aparecen con efectos opuestos: más segundos empuja hacia el abandono y más frecuencia lo aleja. Por eso no conviene leer cada una por separado.

14.4 El umbral

Se calculan las probabilidades fuera de pliegue con las diez particiones y se recorren cortes de 0,05 a 0,95. Se elige el de mayor Recall con precisión sobre el piso; si varios cortes dan el mismo Recall, el más alto. El umbral elegido es 0,34: con las predicciones fuera de pliegue da un Recall de 0,9722 y una precisión de 0,6034, justo sobre el piso. Con el corte de 0,5 serían 0,9596 y 0,6750: bajar el corte compra 1,3 puntos de Recall a cambio de 7,2 puntos de precisión.

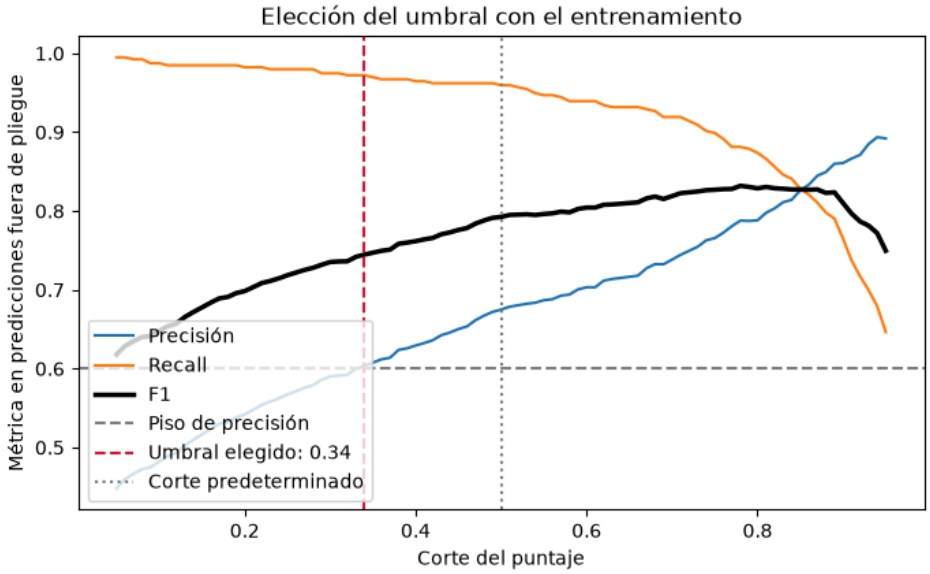


Figura 4. Precisión, recall y F1 según el corte, con predicciones fuera de pliegue del entrenamiento.

14.5 Evaluación final

Como en el cuaderno original, el modelo y el umbral están fijados con entrenamiento y la prueba se mide una sola vez. Este conjunto ya se había consultado en el cuaderno original, así que la medición no es una prueba nueva intacta.

Tabla 23. Métricas del modelo final del cuaderno de corrección en la muestra de prueba (630 clientes, 99 abandonos).

Métrica	Umbral elegido (0,34)	Corte predeterminado (0,50)
Recall	0,9596	0,9394
Precisión	0,6835	0,7440
F1	0,7983	0,8304
Exactitud	0,9238	0,9397
ROC AUC	0,9801	0,9801
Precisión promedio	0,9091	0,9091

Con el umbral de 0,34 se detectan 95 de los 99 abandonos y quedan 4 sin detectar, con 44 falsas alarmas entre los 531 clientes que permanecen: 139 alertas en total. El Recall de prueba (0,9596) coincide con el de la validación anidada (0,9546, desviación de 0,0260) y la precisión (0,6835) con la anidada (0,6684), así que no hay señal de que la elección se ajustara a la prueba. Con el corte de 0,5 el Recall baja a 0,9394 y la precisión sube a 0,7440: bajar el umbral gana dos abandonos.

Tabla 24. Qué pasa si solo se contacta a una fracción de los clientes de mayor riesgo (cuaderno de corrección).

% contactado	Clientes	Abandonos dentro	Precisión	Recall	Veces mejor que al azar
5 %	32	32	100,00 %	32,32 %	6,36×
10 %	63	57	90,48 %	57,58 %	5,76×
15 %	94	79	84,04 %	79,80 %	5,35×
20 %	126	93	73,81 %	93,94 %	4,70×
25 %	158	96	60,76 %	96,97 %	3,87×
30 %	189	97	51,32 %	97,98 %	3,27×

Contactando al 5 % de los clientes (32) los 32 son abandonos; con el 20 % (126) se alcanza el 93,9 % de los abandonos, y con el 25 % (158) el 97,0 % con precisión de 0,6076.

15. Comparación de los dos cuadernos

Los dos modelos se miden sobre los mismos 630 clientes de prueba.

Tabla 25. Resultados de los dos cuadernos en la muestra de prueba.

	Cuaderno original	Cuaderno de corrección
Modelo	Random Forest, peso por clase	Red neuronal, doce variables
Umbral	0,50	0,34
Validación (entrenamiento)	F1 0,8742 con precisión 0,8463 y recall 0,9040 (selección del umbral)	Recall 0,9546 con precisión 0,6684 (validación anidada)
Recall en prueba	0,9495	0,9596
Precisión en prueba	0,8624	0,6835
F1 en prueba	0,9038	0,7983
Exactitud en prueba	0,9683	0,9238
Abandonos detectados	94 de 99	95 de 99
Falsas alarmas	15	44
Clientes alertados	109	139

Los dos cuadernos coinciden en lo importante. En ambos, quitar variables o resumirlas con PCA no mejora al modelo elegido: en el original, ANOVA empeora a los tres candidatos; en el de corrección, ninguna de las 180 combinaciones supera al mejor modelo con las doce variables. En ambos, la señal está en combinaciones de variables y gana un modelo no lineal. Y en ambos el resultado de prueba cae dentro del rango que anticipaba la validación.

La diferencia está en el compromiso entre detectar y molestar. La corrección detecta un abandono más (95 contra 94), pero con 44 falsas alarmas en lugar de 15, casi el triple. Con 99 abandonos en la prueba, un cliente más o menos es un punto de Recall, así que la ganancia en detectados está dentro del ruido; el costo en falsas alarmas no. El original queda mejor en precisión (0,8624 contra 0,6835) y en F1 (0,9038 contra 0,7983). El Recall con piso de 0,60 es por construcción más permisivo con las falsas alarmas: es la elección correcta si perder a un cliente cuesta mucho más que una llamada de más, y esa relación de costos no se estimó en este trabajo.

16. Despliegue: variables, modelos guardados y aplicación web

16.1 En qué se apoya el modelo

Random Forest permite ver cuánto usó cada variable para separar a los clientes, sumando los 400 árboles. No dice si empuja hacia el abandono o en contra, solo cuánto se apoyó en ella. (La sección 14.3 da el equivalente para el modelo del cuaderno de corrección, con la dirección de cada efecto.)

Tabla 26. Importancia de cada variable en el Random Forest del cuaderno original.

Variable	Importancia
`estado`	**0,1687**
`segundos_uso`	**0,1411**
`frecuencia_uso`	**0,1392**
`quejas`	**0,1244**
`antiguedad`	**0,1007**
`valor_cliente`	**0,0949**
`numeros_distintos`	**0,0677**
`frecuencia_sms`	**0,0626**
`fallas_llamadas`	**0,0420**
`grupo_edad`	**0,0395**
`nivel_cargo`	**0,0177**
`plan_tarifario`	**0,0015**

Hay dos cosas que no coinciden con lo que mostró la exploración. `quejas` era la señal más fuerte por phik y por ANOVA, y aquí queda cuarta: es una variable binaria que solo afecta al 8 % de los clientes, así que el modelo la usa en pocos cortes, aunque sean muy decisivos. Y `antiguedad`, que por ANOVA casi no distinguía, aparece quinta porque el bosque la aprovecha combinada con las demás, otra forma de ver lo que ya mostró la sección 7.4. Al final quedan `nivel_cargo` y `plan_tarifario`, que el modelo prácticamente no usa.

16.2 Modelos guardados

Cada cuaderno guarda su modelo en un solo archivo con tres cosas: el modelo ya entrenado (el `Pipeline` completo), el umbral y los nombres de las doce columnas en el orden en que las espera. Aparte va un archivo de metadatos en JSON con las decisiones, las

cifras de validación y de prueba y las versiones de las librerías. En ambos casos se comprobó que, al recargar el archivo, el modelo predice exactamente las mismas probabilidades y las mismas alertas sobre la prueba.

- El cuaderno original guarda `models/modelo_final.joblib` y `models/metadata_modelo.json`, y escribe `resultados/priorizacion_test.csv` y los conjuntos `data/train.csv` y `data/test.csv`.
- El cuaderno de corrección guarda `models/modelo_recall.joblib` y `models/metadata_recall.json` y `resultados/priorizacion_recall.csv`. Agrega una función `predecir_abandono` que valida que lleguen las doce columnas numéricas antes de predecir, y guarda en los metadatos los cortes de referencia para calcular el PSI (índice de estabilidad de la población) de lotes futuros. Con la prueba como lote de ejemplo, el PSI es menor a 0,03 en las doce variables, como era de esperar al venir de la misma partición aleatoria; sirve como comprobación del cálculo y no como señal de desempeño.

16.3 Aplicación web

El modelo del cuaderno original se publicó en una aplicación construida con Gradio (Abid et al., 2019), que carga el modelo exportado y reconstruye ella misma la partición 80/20 (misma semilla) a partir del archivo original, sin depender de archivos intermedios. Se ejecuta en local con `uv run python app.py` y está preparada para ejecutarse en Google Colab. Tiene tres pestañas:

- **Evaluar un cliente.** Un formulario con las doce entradas del modelo, cada una con su descripción. Muestra el riesgo estimado en porcentaje, un veredicto según el umbral del modelo (0,50) y los factores que más empujan el riesgo de ese cliente en concreto, comparando cada entrada contra la mediana de entrenamiento. Botones de ejemplo cargan un cliente tranquilo, uno justo en el límite del umbral, uno en riesgo alto o uno real al azar de la muestra de prueba, mostrando en ese último caso qué ocurrió de verdad.
- **Priorizar contactos.** Recibe un archivo CSV con un cliente por fila (en español o con los nombres originales en inglés de UCI) y devuelve una tabla ordenada por riesgo, con un resumen y un archivo descargable.
- **Sobre el modelo.** Muestra el modelo, la estrategia de balanceo, la calibración (ninguna), el umbral, las métricas de prueba y la de validación, en qué se apoya el modelo (importancia de variables) y un deslizador en vivo que recalcula cuántos abandonos se cubren según cuántos clientes se contacten, sobre los 630 clientes reales de prueba.

La aplicación incluye un aviso visible de que es un proyecto académico, que no estima si una oferta concreta evitará el abandono ni cuánto ahorraría hacerlo, y que no reemplaza el criterio del equipo de retención.

17. Conclusiones, limitaciones y trabajo futuro

17.1 Conclusiones

- **El desbalance define la métrica.** Con 15,71 % de abandono, un modelo con F1 de 0 puede tener una exactitud de 84,29 %, y uno que alerta a todos tiene Recall de 1,00. El cuaderno original optimiza el F1; el de corrección, el Recall con un piso de precisión de 0,60.
- **Casi todas las entradas originales se conservan.** De trece variables solo se elimina `edad`, por ser idéntica a `grupo_edad`. `estado` y `valor_cliente` se conservan después de comprobar que aportan, aunque su definición exacta no esté documentada.
- **La señal es fuerte y no es lineal.** Con phik de 0,734, las quejas son la variable más asociada al abandono, seguida de `estado` (0,705). La señal está en combinaciones de variables, por lo que los árboles, los ensambles y la red neuronal superan con claridad a los modelos lineales.
- **Ajustar los hiperparámetros fue lo que más movió el resultado.** Con una búsqueda de 60 combinaciones por modelo, seis modelos superan 0,95 de Recall con la precisión sobre el piso; la SVM pasó de 0,4147 a 0,9546 y la red neuronal de 0,6314 a 0,9597 respecto a sus valores por defecto.
- **Quitar variables, seleccionarlas o resumirlas no mejoró al modelo elegido en ninguno de los dos cuadernos.** ANOVA empeora a los tres candidatos del original y, en la corrección, la mejor de las 180 combinaciones es la de las doce variables. PCA además haría imposible explicar el riesgo variable por variable.
- **Los modelos son útiles para priorizar contactos.** El original detecta 94 de 99 abandonos de la prueba con 15 falsas alarmas; contactando al 15 % de mayor riesgo se cubre el 86 % de los abandonos con 90 % de precisión. La corrección detecta 95 con 44 falsas alarmas.
- **Los resultados se reportan con su margen.** Con 99 abandonos en la prueba, cada cliente pesa un punto de Recall y la diferencia entre los dos modelos en detectados cabe en ese margen. En el original, el F1 de validación (0,8722) y el de prueba (0,9038) piden decir que el desempeño está entre 0,87 y 0,90.

17.2 Limitaciones

- **Sin fechas ni identificador de cliente.** No se puede comprobar el orden exacto de los hechos cliente por cliente, ni descartar que alguna entrada (por ejemplo, uso igual a cero) ya reflejara un abandono ocurrido.
- **Perfiles posiblemente compartidos entre entrenamiento y prueba.** La partición no agrupa perfiles idénticos antes de partir (sección 3); no se midió cuántos clientes con el mismo perfil de entrada quedaron repartidos entre los dos conjuntos.
- **La prueba ya se había consultado.** Los dos cuadernos reutilizan el mismo conjunto de prueba, que ya se había consultado en una versión anterior; sus resultados no se usaron

para elegir nada, pero no son una prueba nueva intacta.

- **Selección de variables fuera de la validación anidada.** En el cuaderno de corrección, el método de selección y el número de variables se eligieron con todo el entrenamiento, y la validación anidada no los corrige. El umbral de 0,34 deja la precisión del entrenamiento justo sobre el piso (0,6034).
- **`estado`, una de las variables más usadas, es un indicador tardío.** Suele cambiar cuando el cliente ya dejó de usar el servicio, justo cuando queda menos margen para retenerlo.
- **Datos de una sola empresa.** No se sabe cómo se comportarían los modelos en otro periodo, otro mercado o con otra definición de abandono.
- **El umbral es una decisión de negocio.** Los umbrales de 0,50 y 0,34 salen de optimizar una métrica; el adecuado depende de cuánto cueste contactar a un cliente frente a perderlo, y ese costo no se estimó en este trabajo.
- **No se estima el efecto de una intervención.** Los modelos ordenan a los clientes por riesgo de abandono; no estiman si una oferta de retención cambiaría ese riesgo ni cuánto ahorraría.

17.3 Trabajo futuro

- Conseguir datos con fecha para validar con una partición temporal y descartar por completo que alguna entrada refleje el desenlace.
- Medir el solapamiento de perfiles entre entrenamiento y prueba, y comparar contra una partición que agrupe perfiles idénticos.
- Costos asimétricos entre contactar de más y perder un cliente, y selección del umbral a partir de esos costos; con ellos, decidir entre los dos modelos.
- Un diseño experimental (A/B) que mida si contactar a los clientes de mayor riesgo reduce el abandono real.
- Datos de otra empresa o periodo, y despliegue permanente de la aplicación.

18. Referencias

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. y Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv:1906.02569*.

Baak, M., Koopman, R., Snoek, H. y Klous, S. (2020). A new correlation coefficient between categorical, ordinal and interval variables with Pearson characteristics. *Computational Statistics & Data Analysis, 152*, 107043.

Bergstra, J. y Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

Lemaître, G., Nogueira, F. y Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research, 18*(17), 1–5.

Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*, 4765–4774.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Varma, S. y Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics, 7*, 91.

Iranian Churn Dataset (2020) [Conjunto de datos]. UCI Machine Learning Repository, identificador 563. https://doi.org/10.24432/C5JW3Z

Anexo. Reproducibilidad y estructura del repositorio

El entorno se gestiona con uv (`pyproject.toml` y `uv.lock`, Python 3.11) y los cuadernos también están pensados para correr en Google Colab. Para reproducir los resultados basta con clonar el repositorio, ejecutar `uv sync` y correr el cuaderno que se quiera revisar; no dependen de ningún archivo intermedio, solo del CSV original de UCI. El cuaderno de corrección guarda los resultados de la búsqueda y de la matriz en `resultados/`; si esos archivos existen los reutiliza y la ejecución completa toma del orden de media hora, y desde cero toma más de una hora.

Tabla 27. Archivos principales del repositorio.

Archivo	Responsabilidad	Genera
`Prediccion_abandono_clientes_Original.ipynb`	Análisis original, de principio a fin: exploración, preparación, 41 combinaciones, ajuste de tres candidatos, umbral, evaluación y despliegue	`data/train.csv`, `data/test.csv`, `models/modelo_final.joblib`, `models/metadata_modelo.json`, `resultados/priorizacion_test.csv`
`Prediccion_abandono_clientes_Correccion.ipynb`	Corrección pedida por la profesora: ajuste de los doce modelos, matriz de 180 combinaciones con selección, PCA y SHAP, validación anidada, umbral por Recall con piso y despliegue	`resultados/ajuste_hiperparametros.csv`, `resultados/matriz_combinaciones.csv`, `resultados/curvas_k.csv`, `resultados/priorizacion_recall.csv`, `models/modelo_recall.joblib`, `models/metadata_recall.json`
`app.py`	Aplicación Gradio que publica el modelo del cuaderno original	—
`presentacion/presentacion.html`	Presentación interactiva del análisis original	—

Los modelos entrenados (`models/*.joblib`) no se versionan: pesan cerca de 50 MB y se regeneran ejecutando el cuaderno correspondiente. La aplicación necesita `models/modelo_final.joblib`, que genera el cuaderno original.